Kubernetes supply chain and runtime security

Supply chain threats and SLSA, image scanning, SBOMs and cosign signing, admission enforcement of provenance, Falco runtime detection, seccomp, AppArmor and sandboxed runtimes.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. The software supply chain threat model and SLSA (12 min)
2. Image scanning and vulnerability management (12 min)
3. SBOMs, signing and verification with cosign (12 min)
4. Admission enforcement of provenance with policy (12 min)
5. Runtime detection with Falco and audit events (12 min)
6. Sandboxing: seccomp, AppArmor and runtime classes (12 min)
Cheat sheet

Lesson 1. The software supply chain threat model and SLSA

12 minutes. Map where attackers can interfere between source code and a running pod, and use SLSA levels to decide which controls to add first.

Most cluster compromises start before the cluster: in a dependency, a build server or a registry. A supply chain threat model lists every step from commit to container and what can go wrong at each.

What you will learn

- The stages from source to running pod
- Typical attacks at each stage
- What SLSA levels ask for
- Provenance and why it matters
- Choosing the first controls

The stages

Code is written and reviewed, dependencies are pulled, a build runs, an image is pushed to a registry, a deployment tool pulls it and the cluster runs it. Each hand over is a place where something can be swapped, so each needs a check.

Typical attacks

Examples include a malicious or hijacked dependency, a stolen CI token that pushes a poisoned image, a tag that is moved to a different image, a compromised build runner and an unreviewed change merged by a bot. Attackers prefer places with weak access control and little logging.

SLSA levels

SLSA, the supply chain levels for software artifacts, describes how trustworthy a build is. Level one asks for provenance that records how an artifact was built. Higher levels ask for builds on a hosted service that signs provenance, with isolation so a user cannot forge it. The levels give a roadmap instead of a single yes or no.

Provenance

Provenance is a signed statement: this image digest was built from this commit by this pipeline at this time. A cluster that checks provenance can refuse an image built by an unknown pipeline even if the registry contains it.

First controls

Pin images by digest, scan dependencies, require reviews, protect the main branch and use short lived credentials in CI. Then sign images and verify at admission. Start with the steps that block the most likely attack for you.

Diagram: The software supply chain threat model and SLSA

Part	What it does	What goes wrong
Source and dependencies	Code and packages.	Hijacked package.
Build pipeline	Produces the image.	Stolen token.
Registry	Stores images.	Moved tag.
Cluster	Runs the image.	Unverified image.

Flow: Source and dependencies to Build pipeline / Build pipeline to Registry / Registry to Cluster

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The tag that changed after review

Incident: *A deployment pulled a latest tag that pointed to an image nobody had reviewed.*

```
$ kubectl get pod web-1 -o jsonpath='{.status.containerStatuses[0].imageID}'
registry.example.com/web@sha256:9f2a77c1
$ crane digest registry.example.com/web:1.8
sha256:41be0d20
(this is a simulated illustration, not a real incident)
```

The tag 1.8 had been moved to a new image after the review, so the running digest did not match the reviewed one. The team pinned deployments by digest, made tags immutable in the registry and added a check that compares the deployed digest to the one in the release record.

Common mistakes

- Deploying mutable tags such as latest
- Long lived registry credentials in CI
- Trusting any image in the registry
- No record of what built an image
- Starting with signing before basic access control

Check yourself

What does provenance state?

Answer: How an artifact was built and from what

Why pin images by digest?

Answer: A digest identifies exact content, a tag can move

What is SLSA?

Answer: A framework of levels for build and artifact integrity

Which is a supply chain attack?

Answer: A poisoned dependency pulled into the build

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Image scanning and vulnerability management

12 minutes. Scan images in CI and in the registry, read results without panic, prioritise by exploitability and set an expiry for exceptions.

A scanner reports hundreds of findings for almost any image. The skill is to cut noise, decide what to fix first and keep a record of what you accepted and why.

What you will learn

- What scanners check and what they miss
- Reducing findings with small, current base images
- Prioritising by severity, exploitability and exposure
- Gates in CI and continuous rescans
- Managing exceptions with VEX and expiry

What scanners check

Tools such as Trivy compare package versions in the image to vulnerability databases. They also flag misconfigurations and secrets in files. They do not find unknown flaws or logic errors, and a clean result today may not be clean next week as new vulnerabilities are published.

Smaller images

Most findings come from the operating system packages in the base image. A minimal or distroless base, multi-stage builds and regular rebuilds remove many findings without any code change. Fewer packages also mean a smaller attack surface.

Prioritising

Severity alone is not enough. Ask whether the vulnerable package is loaded, whether a fix exists, whether a public exploit is known and whether the workload is reachable from the internet. A critical issue in an unused tool in a build image matters less than a high severity issue in an exposed service.

Gates and rescans

Fail a pipeline on fixable critical and high findings that apply to the runtime image. Rescan images already in the registry on a schedule, because new vulnerabilities appear after the build. Tell owners through tickets with the image and the fix version.

Exceptions

When you cannot fix something, record the reason, the owner and an expiry date. A VEX statement can state that a vulnerability is not exploitable in this product. Reviewing expiring exceptions keeps the list honest.

Diagram: Image scanning and vulnerability management

Part	What it does	What goes wrong
CI scan	Gates the build.	Ignored failures.
Registry rescan	Finds new issues.	No owner.
Triage	Prioritise and assign.	Severity only.
Rebuild and deploy	Ships the fix.	Slow rollout.

Flow: CI scan to Registry rescan / Registry rescan to Triage / Triage to Rebuild and deploy

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A critical finding two weeks after release

Incident: *A library vulnerability was published after the image shipped and nobody noticed.*

```
$ trivy image --severity CRITICAL,HIGH --ignore-unfixed registry.example.com/api@sha256:41be0d20
Total: 2 (HIGH: 1, CRITICAL: 1)
libssl3  CVE-2026-0001  CRITICAL  3.0.11-1  3.0.13-1
(this is a simulated illustration, not a real incident)
```

The image had no scheduled rescan, so the new finding only appeared during a manual check. A nightly rescan job now opens a ticket for the owning team, and the base image is rebuilt weekly so most fixes arrive without effort.

Common mistakes

- Failing builds on every low severity finding
- Scanning only at build time
- Ignoring whether a fix exists
- Exceptions with no expiry
- Big images with shells and package managers

Check yourself

Why rescan images in the registry?

Answer: New vulnerabilities are published after the build

What reduces findings most cheaply?

Answer: A smaller, current base image

What should exceptions include?

Answer: Owner, reason and an expiry date

Which finding is most urgent?

Answer: A fixable high severity issue in an internet facing service

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. SBOMs, signing and verification with cosign

12 minutes. Generate a software bill of materials, sign images and attestations with cosign and keyless identities, and verify them before use.

A signature says who vouched for an image. A software bill of materials says what is inside it. Together they let you answer which images contain a bad library and prove who built them.

What you will learn

- What an SBOM contains and how to use it
- Signing images and attaching attestations
- Keyless signing with identity and a transparency log
- Verifying signatures in CI and in the cluster
- Key and identity management

SBOMs

An SBOM lists the packages and versions in an image, in formats such as SPDX or CycloneDX. Generate it at build time, store it with the image and search it when a new vulnerability is announced. You can then list affected images in minutes instead of rescanning everything.

Signing

Cosign signs the image digest and stores the signature next to the image in the registry. An attestation is a signed statement about the image, such as its SBOM or its provenance. Sign the digest, never the tag, since a tag can move.

Keyless signing

Keyless signing uses a short lived certificate issued to the identity of the CI job, such as a workflow in a repository, through OpenID Connect. The signing event is recorded in a public transparency log. You do not store a long lived key, and a verifier checks the identity and the issuer instead of a key.

Verifying

Verification checks that a signature exists for the digest and that the signer identity and issuer match what you expect. Do it in the pipeline before promotion and again in the cluster at admission. A signature from the wrong identity must fail.

Managing identities

If you use keys, keep them in a KMS or hardware module and rotate them. With keyless signing, protect the CI workflow itself, because whoever can run it can sign. Restrict which branches and workflows are allowed to produce releases.

Diagram: SBOMs, signing and verification with cosign

Part	What it does	What goes wrong
Build job	Builds and generates SBOM.	Open to any branch.
cosign sign	Signs digest.	Wrong identity.
Registry	Image and signature.	Tag moved.
Verify step	Checks identity.	Not enforced.

Flow: Build job to cosign sign / cosign sign to Registry / Registry to Verify step

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A signed image from the wrong workflow

Incident: *An image signed by a test workflow was promoted to production.*

```
$ cosign verify registry.example.com/api@sha256:41be0d20 --certificate-identity-regexp 'https://gi \
    thub.com/example/.*' --certificate-oidc-issuer https://token.actions.githubusercontent.com | h \
    ead -c 120
Verification for registry.example.com/api@sha256:41be0d20 --
The following checks were performed on each of these signatures:
(this is a simulated illustration, not a real incident)
```

The identity pattern accepted any workflow in the organisation, including a test workflow. The policy was tightened to the release workflow on the main branch, and the verification step now fails for any other identity.

Common mistakes

- Signing tags instead of digests
- Accepting any identity from the right issuer
- No SBOM stored for later search
- Long lived signing keys in CI variables
- Verifying only in CI and never at admission

Check yourself

What does an SBOM list?

Answer: Packages and versions inside an image

What does cosign sign?

Answer: The image digest

What does keyless signing remove?

Answer: Long lived signing keys to store

What must verification check?

Answer: The signer identity and issuer

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Admission enforcement of provenance with policy

12 minutes. Enforce signed and approved images at admission with Kyverno or similar tools, roll out in audit mode first and design for the unreachable policy engine case.

Verification in CI is advisory until the cluster refuses anything that skipped it. Admission policy turns your supply chain rules into a gate that every workload must pass.

What you will learn

- What an image policy checks
- Kyverno and similar engines
- Audit mode before enforce
- Exceptions and break glass access
- Availability of the policy engine itself

What to check

Useful rules include: images must come from approved registries, must be referenced by digest, must have a valid signature from the release identity and must have an SBOM or provenance attestation. Keep rules small and clear so teams can understand a rejection.

Engines

Kyverno expresses policies as Kubernetes resources and has built in image verification. Other options are OPA Gatekeeper, the built in ValidatingAdmissionPolicy for simple rules and the Sigstore policy controller. Choose by the team skills and by which checks you need, such as signature verification.

Audit first

Start in audit mode, which records violations without blocking. Review the report, fix or exempt each case, then switch namespaces to enforce one by one. A sudden global enforce is the quickest way to stop deployments across the company.

Exceptions

Some system namespaces and vendor images need exceptions. Write them as explicit, reviewed rules with an owner and expiry. Provide a break glass path, such as an annotated exemption with an audit trail, for real emergencies.

Engine availability

A policy webhook with failure policy fail blocks changes when it is down, and with ignore it lets everything through. Run several replicas, spread them across zones, exclude the policy namespace and alert on webhook latency and errors. Decide the failure mode on purpose.

Diagram: Admission enforcement of provenance with policy

Part	What it does	What goes wrong
Deploy request	Applies a pod.	Unsigned image.
API server	Calls the webhook.	Webhook timeout.
Policy engine	Verifies signature.	Down or slow.
Allow or deny	Returns the decision.	Wrong failure mode.

Flow: Deploy request to API server / API server to Policy engine / Policy engine to Allow or deny

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Enforce mode that blocked a security patch

Incident: *A fix could not be deployed because its new image was not signed yet.*

```
$ kubectl apply -f deploy.yaml
Error from server: admission webhook "verify-images.kyverno.svc" denied the request: image registr \
    y.example.com/api@sha256:77aa verification failed: no signatures found
$ kubectl get cpol verify-images -o jsonpath='{.spec.validationFailureAction}'
Enforce
(this is a simulated illustration, not a real incident)
```

The build pipeline had signed images only on the main branch, but the patch came from a release branch. The pipeline was fixed to sign release branches, and the team used a time limited exception to ship the urgent patch while the fix was in review.

Common mistakes

- Switching to enforce without an audit period
- Rules nobody can read or explain
- No exception process for emergencies
- Running one policy engine replica
- Not excluding the policy engine own namespace

Check yourself

Why start in audit mode?

Answer: To see violations without blocking deployments

What should policy require about references?

Answer: Images by digest

What is a risk of failure policy fail?

Answer: Deployments stop if the engine is down

What belongs in an exception?

Answer: Owner, reason and expiry

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Runtime detection with Falco and audit events

12 minutes. Detect suspicious behaviour in running pods using Falco rules and Kubernetes audit events, tune noise and route alerts with context.

Prevention fails sometimes. Runtime detection watches what containers do and what is done through the API, so unusual behaviour such as a shell in a production pod raises an alert.

What you will learn

- What Falco observes and how rules work
- Useful default detections
- Kubernetes audit events as a second source
- Tuning rules to reduce false alerts
- Alert routing and response

How Falco works

Falco watches system calls through an eBPF probe or a kernel module and matches them against rules. A rule has a condition, an output message and a priority. Because it sees real behaviour, it catches what static scans cannot, such as a process opening a reverse shell.

Useful detections

Start with rules for a shell started inside a container, a write below system directories, reading of service account tokens by unexpected processes, outbound connections to unusual ports and use of package managers in running containers. These are rare in healthy workloads.

Audit events

The API server audit log records who did what through the API: secret reads, exec into pods, role changes. Rules or a log pipeline can alert on a spike of secret reads or on a binding to cluster admin. Together with Falco this covers both inside and outside the container.

Tuning

Every cluster has legitimate shells and package managers, such as debug sessions or build jobs. Add exceptions by namespace, image or process name rather than disabling a rule. Track the share of alerts that were real and revisit the noisy ones weekly.

Routing and response

Send alerts with the pod, namespace, image and rule name to a channel or ticket system. Define what to do for each priority: for example a shell in production triggers quarantine through a network policy label and a review of the audit log.

Diagram: Runtime detection with Falco and audit events

Part	What it does	What goes wrong
Kernel events	System calls.	Probe missing.
Falco rules	Match behaviour.	Noisy rules.
Alert pipeline	Adds context, routes.	No owner.
Responder	Contains and investigates.	Late response.

Flow: Kernel events to Falco rules / Falco rules to Alert pipeline / Alert pipeline to Responder

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A shell in the payments pod

Incident: *Falco reported a shell started in a production container at night.*

```
$ kubectl logs -n falco ds/falco | grep payments | head -2
{"priority":"Warning","rule":"Terminal shell in container","output":"A shell was spawned in a cont \
    ainer (user=root pod=payments-7d9 image=registry.example.com/payments:2.4 command=sh -c id)"}
$ kubectl get events -n payments --field-selector involvedObject.name=payments-7d9 | tail -2
(this is a simulated illustration, not a real incident)
```

The shell ran from a vulnerable endpoint that accepted a command. The pod was quarantined with a deny network policy, evidence was copied, the image was patched and a rule for outbound connections from this namespace was added. The alert reached the responder within a minute because it carried the pod and image names.

Common mistakes

- Disabling a noisy rule instead of adding a scoped exception
- Alerts with no context
- No plan for what a responder does
- Relying only on image scans
- Not enabling audit logging on the API server

Check yourself

What does Falco watch?

Answer: System calls made by containers

What do audit events record?

Answer: Requests made to the API server

How should noise be handled?

Answer: Scoped exceptions and regular tuning

What should an alert include?

Answer: Pod, namespace, image and rule

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Sandboxing: seccomp, AppArmor and runtime classes

12 minutes. Reduce what a compromised container can do with seccomp profiles, AppArmor, dropped capabilities and sandboxed runtimes such as gVisor.

Containers share the host kernel, so a flaw in the kernel can become an escape. Syscall filters and sandboxed runtimes shrink the exposed kernel surface for workloads that need it.

What you will learn

- The shared kernel risk
- Seccomp profiles and RuntimeDefault
- AppArmor and SELinux
- Dropping capabilities and read only root
- RuntimeClass and sandboxed runtimes

The shared kernel risk

A container is a process with namespaces and cgroups. A vulnerability in a system call can let a process escape to the host. Reducing the system calls and privileges available lowers that risk, and strong isolation helps for untrusted code.

Seccomp

Seccomp filters system calls. Setting `seccompProfile: RuntimeDefault` applies the runtime default profile, which blocks several dangerous calls with little breakage. Custom profiles can be recorded from a test run and reviewed. Unconfined is the weakest setting and should not be used.

AppArmor and SELinux

These mandatory access control systems restrict files and operations per process. Use the profiles provided by the runtime or write specific ones for sensitive workloads. Check that your nodes support them, as availability depends on the operating system.

Capabilities and file system

Drop all Linux capabilities and add back only what is needed, run as non-root, disallow privilege escalation and set the root file system read only. Many exploits need a writable path or a capability that a normal service never uses.

Sandboxed runtimes

A RuntimeClass selects a runtime for a pod, for example gVisor or Kata Containers. They add a user space kernel or a lightweight virtual machine, which costs some performance and compatibility but gives much stronger isolation. Use them for untrusted code, multi tenant workloads or build jobs.

Diagram: Sandboxing: seccomp, AppArmor and runtime classes

Part	What it does	What goes wrong
Container process	Makes syscalls.	Over privileged.
Seccomp and MAC	Filters calls.	Unconfined.
Runtime class	Selects sandbox.	Incompatible app.
Host kernel	Shared resource.	Kernel flaw.

Flow: Container process to Seccomp and MAC / Seccomp and MAC to Runtime class / Runtime class to Host kernel

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A seccomp profile that broke a release

Incident: *A custom profile blocked a system call the new library used.*

```
$ kubectl logs api-5c8 | tail -2
Fatal: operation not permitted
$ kubectl get pod api-5c8 -o jsonpath='{.spec.securityContext.seccompProfile}'
{"localhostProfile":"profiles/api.json","type":"Localhost"}
(this is a simulated illustration, not a real incident)
```

The profile had been recorded before a library upgrade and missed a new call. The team switched to RuntimeDefault for the release, re-recorded the custom profile in a test environment and added a test that runs the app with the profile in CI.

Common mistakes

- Leaving seccomp unconfined
- Running privileged containers for convenience
- Writable root file systems
- Adding all capabilities back
- Sandbox runtimes without testing compatibility

Check yourself

What does seccompProfile RuntimeDefault do?

Answer: Applies the runtime default syscall filter

What does a RuntimeClass select?

Answer: The container runtime for a pod

Which setting is risky?

Answer: privileged: true

Why use a sandboxed runtime?

Answer: Stronger isolation for untrusted code

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Supply chain and runtime security

Supply chain controls, scan, sign and verify commands, and pod hardening.

Supply chain controls

Stage	Control
Source	Branch protection, signed commits, reviews
Build	Hosted builder, provenance, pinned bases
Registry	Immutable tags, scanning, signatures
Deploy	Admission policy that verifies signatures
Run	Falco, seccomp, RuntimeClass

Commands

Job	Command
Scan image	`trivy image IMAGE`
Create SBOM	`syft IMAGE -o spdx-json`
Sign	`cosign sign IMAGE@sha256:DIGEST`
Verify	`cosign verify IMAGE --certificate-identity ID --certificate-oidc-issuer URL`

Pod hardening

Setting	Value
seccompProfile	RuntimeDefault
runAsNonRoot	true
allowPrivilegeEscalation	false
runtimeClassName	gvisor for untrusted code

MARGA by EknathaLabs marga.eknathalabs.com